Assignment: Build an MCP Server for a Real Startup

Type: Individual **Timeline:** 2 days **Submit:** GitHub repo link + README

The Idea

Pick a startup, understand what they do, and build an MCP server that would actually be useful to them. Not a toy. Something you could show their team and they'd get why it exists.

Part 1: The Rules

Everything you build has to follow these. We'll check your submission against them.

1. It should work beyond Claude

- Connect your server to at least 2 hosts: Claude Desktop + one more (Cursor, VS Code Copilot, or a local model via Ollama).
- Note what worked differently. Some clients only support tools, not resources or prompts. Smaller models might pick the wrong tool.
- Think about this: do your tool names and descriptions still make sense to a less capable model?

2. It should fit into a bigger workflow

- Your server won't live alone. Explain where it sits in the company's day-to-day work.
- What triggers it? What comes before and after it? Is a human in the loop?
- Local only (stdio) or could a whole team use it (HTTP)? Why?

3. It should be smart with space

- If your use case involves documents, don't dump whole files into context.
- Use RAG inside a tool and return only the relevant chunks.
- Explain your chunk size and how many chunks you return.
- Show rough token counts: dumping the full doc vs. your approach.

Part 2: The Build

Step 1: Pick a company

- Go through YC's directory or any startup list.
- Read their descriptions and pick one.
- Write 2 to 3 lines on what they do and who their users are.
- Just make sure the company you pick hasn't been picked by anyone else.

Step 2: Design

- What problem does your MCP server solve for them? Is it for their internal team or their customers?

deboistech · 2026-10-05

- Which tools, resources, and prompts will you build, and why each one?

Step 3: Build

- Build a working MCP server.
- Use mock data, since you won't have the company's real data.
- Keep it focused. A few good tools beat ten half-working ones.

Step 4: Prove the rules

- Show it running on 2+ hosts.
- Add your notes on all 3 rules from Part 1.

What Your README Should Have

1. Company name + what they do (2 to 3 lines)
2. The problem your server solves
3. Tools, resources, and prompts, and why you chose them
4. How to run it
5. Host comparison: what differed across the 2+ hosts
6. Workflow: where it fits in the company's process
7. Space: token counts before vs. after (if docs are involved)
8. Screenshots or a short demo video

Tips

- Start small. Get one tool working end to end before adding more.
- Write tool descriptions like you're explaining to someone new.
- If you get stuck, look back at the learning log server we built in class.

deboistech